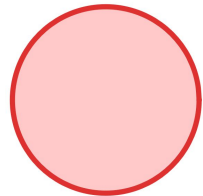




Review and Analysis of Design Patterns in Distributed Systems

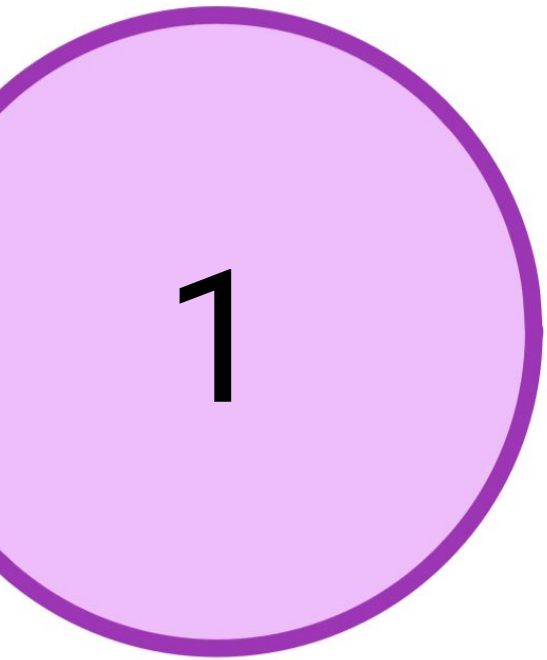


Wiktor Szymański

promotor

dr hab. inż. Anna Kobusińska, prof. PP





1

Motivation

1

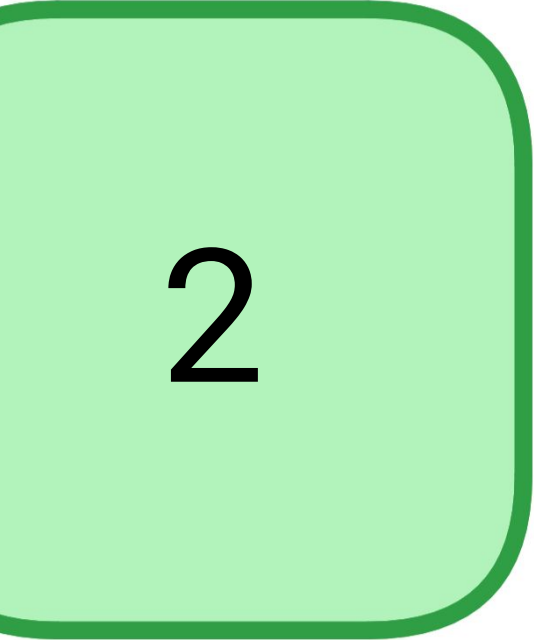
Motivation - Research Questions

Q1 How does the outbox delivery mechanism affect the system's performance, and what restrictions does it impose?

Q2 How does the snapshotting mechanism affect the event sourced system's performance?

Q3 Can a cache on the write side improve the performance of an event sourced system?

Q4 How does enforcing the aggregate boundaries affect database resource contention?



2

Reviewed Patterns

2

Reviewed Patterns

- Transactional Outbox
- Event Sourcing
- Command Query Responsibility Segregation
- Saga



3

Reference System

3

Reference System

Domain:

- Inventory Item Reservation Service
- Two aggregates:
 - Inventory Item
 - Order
- One invariant:
 - Inventory stock cannot be oversold

3

Reference System

Transactional Outbox (T0) system:

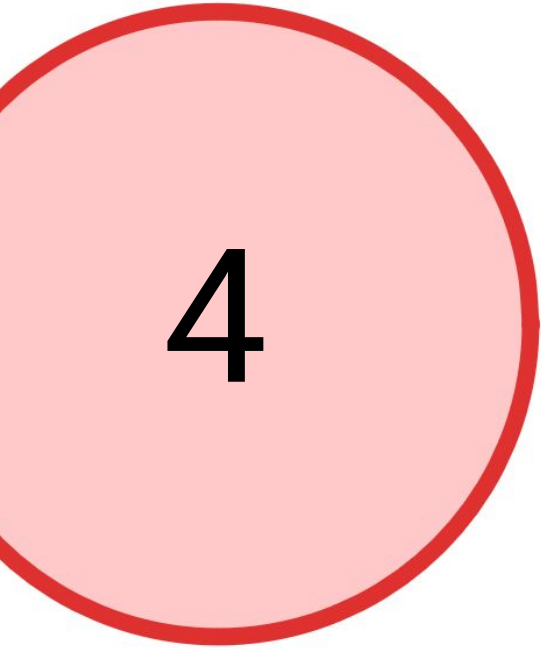
- T0-1 – scheduled poll
- T0-2 – LISTEN/NOTIFY trigger
- T0-3 – after-commit publish

3

Reference System

Event Sourcing (ES) system:

- ES-1 – full event stream replay
- ES-2 – snapshots
- ES-3 – write path aggregate cache



4

Tests

4

Methodology

Environment:

- the same machine
- containers with common resource limits:
 - service 10 CPUs / 12 GiB mem
 - database 10 CPUs / 16 GiB mem

4

Methodology

Load scenarios:

- breakpoint – increasing by 10 req/s every minute
- average-load – stable 60 req/s
- *stress – stable rate around the knee

4

Methodology

Workload variants:

- W-base – 100 distinct items, 4 items per order
- W-hot – 25 distinct items, 4 items per order
- W-fan – 400 distinct items, 16 items per order

4

Methodology

Knee definition:

- 30-second windows
- window is balanced when backlog increases by no more than 1% of incoming rate
- knee value is the last balanced window



5

Results

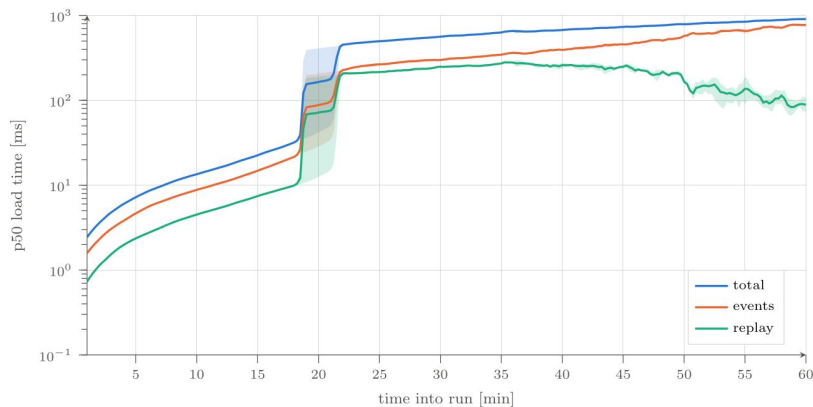
Results - Q1

	TO-1	TO-2	TO-2 with workaround	TO-3
Trigger	scheduled polling every 100 ms	NOTIFY when inserting into outbox table	NOTIFY when flag set, every 10 ms	after-commit phase
Publish lag p50 [ms]	94.1	7.76	19.2	9.90
Optimistic lock retries [1/s]	18.5	0.196	1.85	<0.01
Knee [orders/s]	701	168	658	699

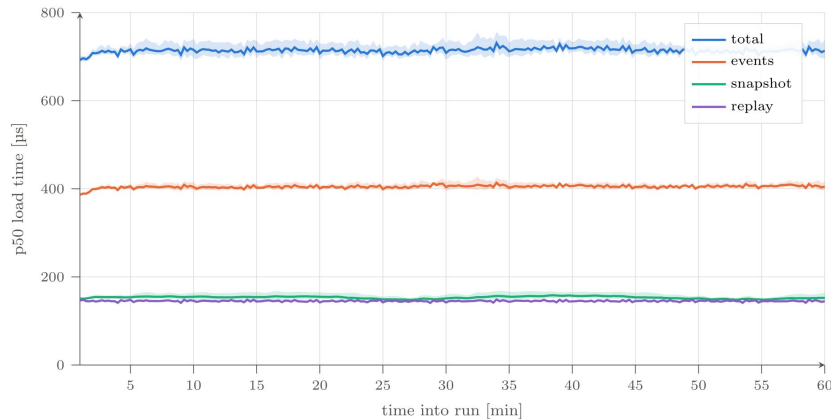
5

Results - Q2

Snapshotting every 100 events limits aggregate rebuilds to a fixed number of events and cuts p50 state load time in the 60th minute from 907 ms to 713 μ s.



no snapshots

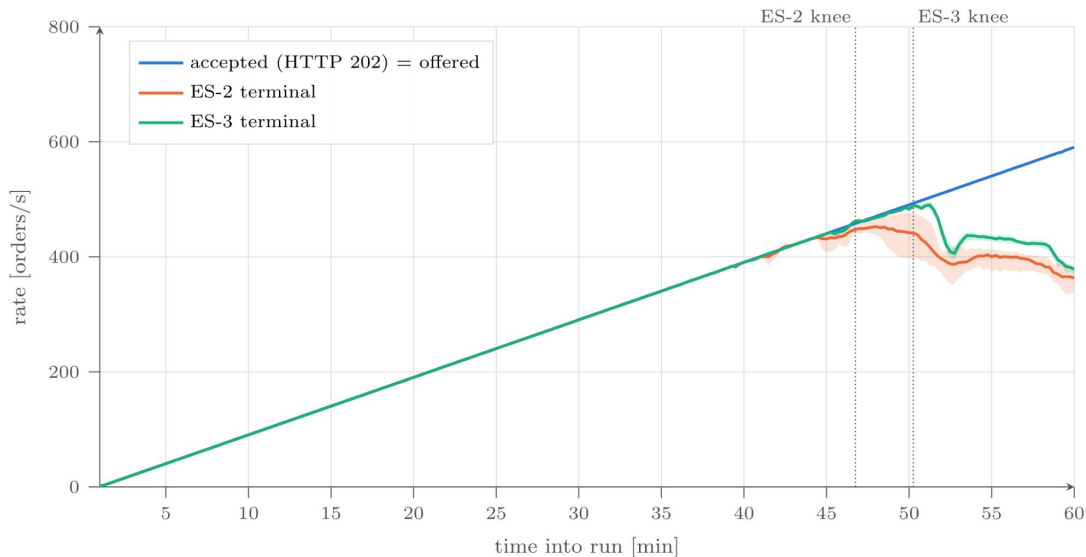


with snapshots

5

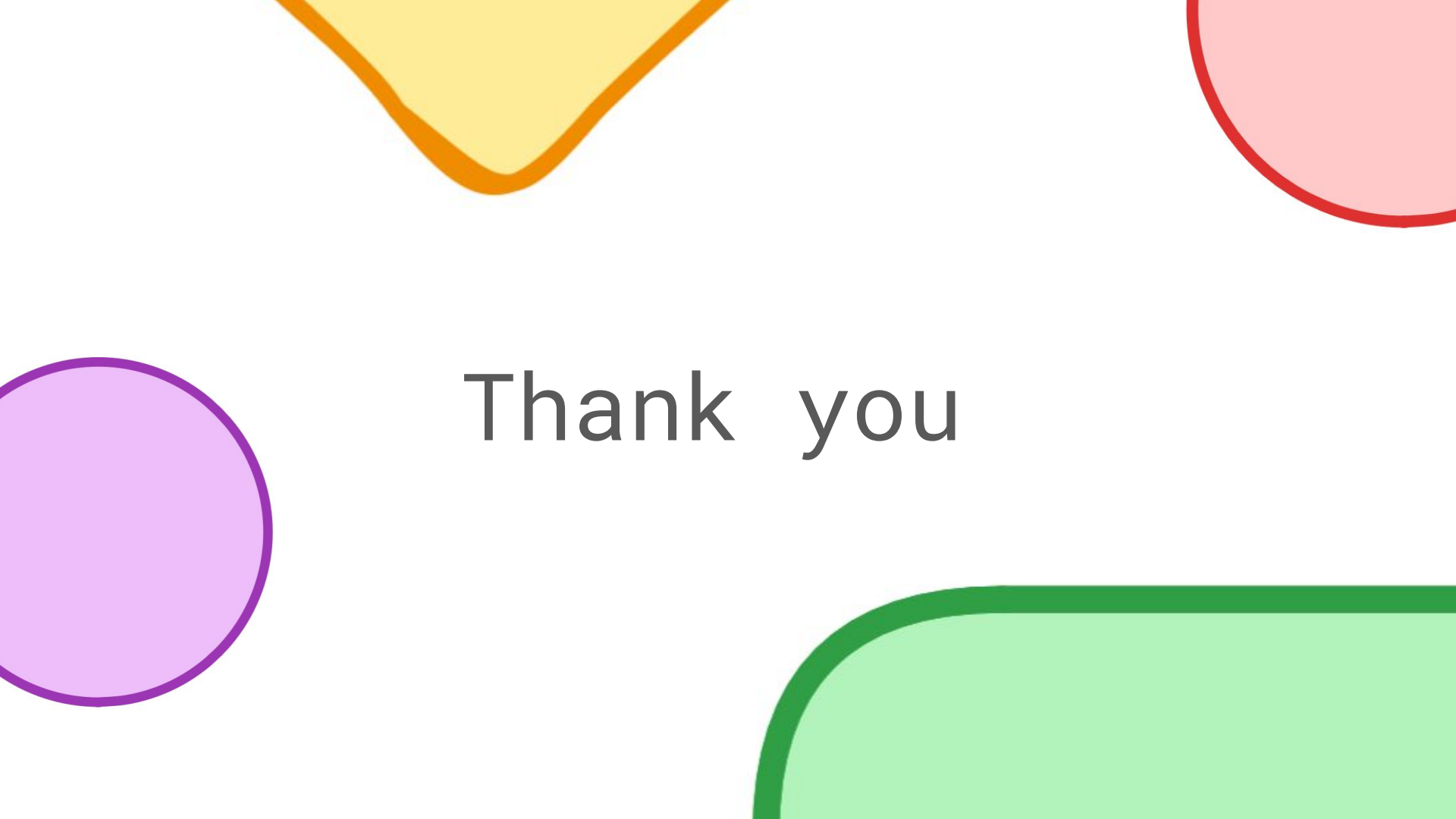
Results - Q3

The write-side cache raised throughput by about 8% and reduced reads from the database, because only the events the cached instance is missing get replayed.



Results - Q4

	Workload	Knee [orders/s]	Change vs W-base	Conflict ratio at 140 orders/s	Rejected orders
TO-3	W-base	699	-	0.012	4.5%
	W-hot	332	-52%	0.041	43%
	W-fan	245	-65%	0.55	29%
ES-2	W-base	465	-	0.016	<0.1%
	W-hot	425	-9%	0.053	7.9%
	W-fan	155	-67%	0.026	0%



Thank you